Լաբորատոր աշխատանք 11

Օբյեկտի փոխանցումը ֆունկցիային

Աշխատանքի նպատակը*:* Տարբերակել օբյեկտի փոխանցումը արժեքով, հղումով և հաստատուն հղումով:

Տեսական մաս*:* Արժեքով փոխանցումը ստեղծում է պարամետրի պատճեն: Հղումով փոխանցումը թույլ է տալիս փոխել սկզբնական օբյեկտը: const հղումը կիրառվում է առանց պատճենման ընթերցելու համար: Արտաքին սովորական ֆունկցիան private տվյալները ստանում է public մեթոդների միջոցով:

Աշխատանքի կատարման կարգը

1. Կարդալ ընդհանուր պահանջները և նշանակված անհատական տարբերակը:
2. Սահմանել դասի դաշտերը, դրանց սահմանափակումները և մեթոդները:
3. Իրականացնել պահանջվող գործողությունները և ստուգել սխալ մուտքերը:
4. Համեմատել ստուգիչ օրինակի արդյունքը ձեռքով ստացված հաշվարկի հետ:

Մեթոդական օրինակ

```
void analyze(const DepositPortfolio& object) {
    object.print();
}
void changeCopy(DepositPortfolio object, int index, double value) {
    object.set(index, value);
    object.print();
}
DepositPortfolio(const DepositPortfolio& other)
    : n(other.n), p(other.p) {
    data = n > 0 ? new double[n] : nullptr;
    for (int i = 0; i < n; i++)
        data[i] = other.data[i];
}
DepositPortfolio reversed(const DepositPortfolio& object) {
    DepositPortfolio result(object);
    result.reverse();
    return result;
}
```

Ընդհանուր պահանջներ

1. Տվյալները դասում պահել private std::vector<double>-ում և p դաշտում: Դինամիկ հիշողության ձեռքով կառավարում այս աշխատանքում չի պահանջվում: Իրականացնել պարամետրերով կոնստրուկտոր, public ընթերցման մեթոդներ, set(index,value) և print(): Յուրաքանչյուր տարբերակի համար իրականացնել ստորև տրված երեք արտաքին ֆունկցիաները՝ առանց friend-ի:
2. Տվյալների ծավալ: Սկզբնական ոչ դատարկ հավաքածուի չափը $3 \le n \le 30$ է: Չափի փոփոխություն պահանջող մեթոդներում թույլատրվում է $0 \le n \le 30$: Բոլոր թվերը վերջավոր են: Հավասար լավագույն արդյունքների դեպքում ընտրել առաջին ինդեքսը: Իրական թվերի հավասարությունը համեմատել 10^{-9} ճշտությամբ:

3. Վավերացում: Սխալ չափով կամ պարամետրերով օբյեկտ չստեղծել. մուտքը կրկնել main()-ում: Թույլատրելիության կանոնները ստուգել նաև փոփոխող մեթոդներում: Սխալ ինդեքսի կամ արժեքի դեպքում վերադարձնել false և պահպանել նախկին տվյալները: Հաշվարկները չպետք է փոփոխեն օբյեկտը:

Ստուգման հարցեր

- Ի՞նչ է տեղի ունենում օբյեկտը արժեքով փոխանցելիս:
- Ի՞նչ տարբերություն կա հղումով և const հղումով փոխանցման միջև:
- Ե՞րբ է կանչվում պատճենող կոնստրուկտորը:
- Ինչպե՞ս կանխել օբյեկտի չնախատեսված փոփոխումը:

Անհատական առաջադրանքներ

№1 Բանկային ավանդների պորտֆել

Տվյալներ: Ստեղծել DepositPortfolio դաս՝ պահելով ավանդների գումարների զանգվածը և տարեկան p տոկոսադրույքը: Ժամկետը մեկ տարի է, տոկոսները պարզ են:

Հաշվարկներ: Հաշվել ընդհանուր տարեկան եկամուտը՝ Σ(amount·p/100), գտնել ամենամեծ վերջնական գումարով ավանդի ինդեքսը և պորտֆելի միջին գումարից մեծ ավանդների քանակը:

Թույլատրելի արժեքներ: բոլոր գումարները >0, 0≤p≤100:

Ֆունկցիաներ: analyze(const DepositPortfolio&) արտածում է երեք հաշվարկները: changeCopy(DepositPortfolio object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const DepositPortfolio&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: amount={100000,250000,150000,300000}, p=8: Ինդեքսները սկսվում են 0-ից:

№2 Բանկային հաշվի գործարքներ

Տվյալներ: Ստեղծել AccountLedger դաս՝ պահելով գործարքների գումարների զանգվածը և p սկզբնական մնացորդը: Դրականը մուտք է, բացասականը՝ ելք:

Հաշվարկներ: Հաշվել վերջնական մնացորդը, գտնել գործարքից հետո ստացված ամենափոքր մնացորդի ինդեքսը և այն գործարքների քանակը, որոնցից հետո մնացորդը բացասական է:

Թույլատրելի արժեքներ: p≥0, գործարքները վերջավոր են: Ուսումնական մոդելում բացասական մնացորդը թույլատրվում է:

Ֆունկցիաներ: analyze(const AccountLedger&) արտածում է երեք հաշվարկները: changeCopy(AccountLedger object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const AccountLedger&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: transactions={5000,-12000,3000,-2000}, p=6000: Ինդեքսները սկսվում են 0-ից:

№3 Վարկերի ամսական տոկոսներ

Տվյալներ: Ստեղծել LoanPortfolio դաս՝ պահելով վարկերի ընթացիկ մայր գումարների զանգվածը և ամսական p տոկոսադրույքը:

Հաշվարկներ: Հաշվել մեկ ամսվա տոկոսների գումարը՝ Σ(debt·p/100), գտնել ամենամեծ տոկոսագումարով վարկի ինդեքսը և մայր գումարների միջինից փոքր վարկերի քանակը:

Թույլատրելի արժեքներ: բոլոր մայր գումարները >0, 0≤p≤100: Սա մեկ ամսվա պարզ տոկոսի մոդել է՝ առանց մարումների և տույժերի:

Ֆունկցիաներ: analyze(const LoanPortfolio&) արտածում է երեք հաշվարկները: changeCopy(LoanPortfolio object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const LoanPortfolio&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: debt={200000,500000,300000,800000}, p=1.5: Ինդեքսները սկսվում են 0-ից:

№4 Արտարժութային հաշիվներ

Տվյալներ: Ստեղծել CurrencyPortfolio դաս՝ պահելով նույն արտարժույթով հաշիվների մնացորդների զանգվածը և մեկ միավորի p դրամ փոխարժեքը:

Հաշվարկներ: Հաշվել դրամով ընդհանուր արժեքը՝ Σ(balance·p), գտնել ամենամեծ դրամային արժեքով հաշվի ինդեքսը և 100000 դրամից մեծ արժեքով հաշիվների քանակը:

Թույլատրելի արժեքներ: բոլոր մնացորդները ≥0, p>0:

Ֆունկցիաներ: analyze(const CurrencyPortfolio&) արտածում է երեք հաշվարկները: changeCopy(CurrencyPortfolio object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const CurrencyPortfolio&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: balance={100,250,500,1000}, p=400: Ինդեքսները սկսվում են 0-ից:

№5 Փոխանցումների միջնորդավճարներ

Տվյալներ: Ստեղծել TransferRegister դաս՝ պահելով փոխանցումների գումարների զանգվածը և p միջնորդավճարի տոկոսը: fee=amount·p/100:

Հաշվարկներ: Հաշվել միջնորդավճարների գումարը, գտնել ստացողին հասնող ամենափոքր գումարով փոխանցման ինդեքսը՝ net=amount-fee, և fee>1000 պայմանով փոխանցումների քանակը:

Թույլատրելի արժեքներ: բոլոր գումարները >0, 0≤p≤100:

Ֆունկցիաներ: analyze(const TransferRegister&) արտածում է երեք հաշվարկները: changeCopy(TransferRegister object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const TransferRegister&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: amount={10000,50000,120000,200000}, p=1: Ինդեքսները սկսվում են 0-ից:

№6 Խնայողության նպատակային հաշիվ

Տվյալներ: Ստեղծել SavingsPlan դաս՝ պահելով ամսական ներդրումների զանգվածը և p նպատակային գումարը: Սկզբնական մնացորդը զրո է:

Հաշվարկներ: Հաշվել ընդհանուր խնայողությունը, գտնել առաջին ամսվա ինդեքսը, երբ կուտակային գումարը հասնում է p-ին, իսկ չհասնելու դեպքում վերադարձնել -1: Հաշվել միջին ներդրումից փոքր ներդրումների քանակը:

Թույլատրելի արժեքներ: բոլոր ներդրումները ≥0, p>0:

Ֆունկցիաներ: analyze(const SavingsPlan&) արտածում է երեք հաշվարկները: changeCopy(SavingsPlan object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const SavingsPlan&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: payments={10000,15000,8000,20000}, p=30000: Ինդեքսները սկսվում են 0-ից:

№7 Խանութի զեղչված գնումներ

Տվյալներ: Ստեղծել ShoppingCart դաս՝ պահելով ապրանքների գների զանգվածը և ընդհանուր p զեղչի տոկոսը: Յուրաքանչյուր ապրանքից գնվում է մեկ հատ:

Հաշվարկներ: Հաշվել վճարման ընդհանուր գումարը՝ Σ(price·(1-p/100)), գտնել ամենաթանկ զեղչված ապրանքի ինդեքսը և 10000 դրամից էժան զեղչված ապրանքների քանակը:

Թույլատրելի արժեքներ: բոլոր գները >0, 0≤p≤100:

Ֆունկցիաներ: analyze(const ShoppingCart&) արտածում է երեք հաշվարկները: changeCopy(ShoppingCart object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const ShoppingCart&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: prices={4000,15000,8000,20000}, p=10: Ինդեքսները սկսվում են 0-ից:

№8 Պահեստի պաշարների վերահսկում

Տվյալներ: Ստեղծել StockRegister դաս՝ պահելով ապրանքների առկա քանակների զանգվածը և յուրաքանչյուր ապրանքի համար p նվազագույն պաշարը:

Հաշվարկներ: Հաշվել պաշարը համալրելու ընդհանուր քանակը՝ Σmax(0,p-quantity), գտնել ամենափոքր առկա պաշարով ապրանքի ինդեքսը և համալրման կարիք ունեցող ապրանքների քանակը:

Թույլատրելի արժեքներ: քանակները ոչ բացասական ամբողջ թվեր են, p-ն դրական ամբողջ թիվ է:

Ֆունկցիաներ: analyze(const StockRegister&) արտածում է երեք հաշվարկները: changeCopy(StockRegister object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const StockRegister&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: quantities={3,12,0,7}, p=10: Ինդեքսները սկսվում են 0-ից:

№9 Գրադարանի ուշացված վերադարձներ

Տվյալներ: Ստեղծել LibraryLoans դաս՝ պահելով գրքերի վերադարձի ուշացման օրերի զանգվածը և մեկ օրվա p դրամ տուգանքը: Զրո օր նշանակում է ժամանակին վերադարձ:

Հաշվարկներ: Հաշվել տուգանքների ընդհանուր գումարը, գտնել ամենաշատ ուշացրած գրքի ինդեքսը և 7 օրից ավելի ուշացված գրքերի քանակը:

Թույլատրելի արժեքներ: օրերը ոչ բացասական ամբողջ թվեր են, p>0:

Ֆունկցիաներ: analyze(const LibraryLoans&) արտածում է երեք հաշվարկները: changeCopy(LibraryLoans object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const LibraryLoans&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: days={0,4,10,2}, p=100: Ինդեքսները սկսվում են 0-ից:

№10 Հյուրանոցի ամրագրումներ

Տվյալներ: Ստեղծել HotelBookings դաս՝ պահելով ամրագրումների գիշերների քանակների զանգվածը և մեկ գիշերվա ընդհանուր p դրամ գինը:

Հաշվարկներ: Հաշվել ամրագրումների ընդհանուր արժեքը, գտնել ամենաերկար ամրագրման ինդեքսը և 3 գիշերից երկար ամրագրումների քանակը:

Թույլատրելի արժեքներ: գիշերների քանակները դրական ամբողջ թվեր են, p>0:

Ֆունկցիաներ: analyze(const HotelBookings&) արտածում է երեք հաշվարկները: changeCopy(HotelBookings object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const HotelBookings&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: nights={2,5,1,4}, p=18000: Ինդեքսները սկսվում են 0-ից:

№11 Աշխատակիցների ժամային վճարներ

Տվյալներ: Ստեղծել PayrollRegister դաս՝ պահելով աշխատակիցների ամսվա աշխատաժամերի զանգվածը և մեկ ժամի p դրամ վճարը:

Հաշվարկներ: Հաշվել աշխատավարձերի ընդհանուր գումարը՝ Σ(hours·p), գտնել ամենաբարձր աշխատավարձ ունեցող աշխատակցի ինդեքսը և 160 ժամից ավելի աշխատած անձանց քանակը:

Թույլատրելի արժեքներ: ժամերը ոչ բացասական իրական թվեր են, p>0:

Ֆունկցիաներ: analyze(const PayrollRegister&) արտածում է երեք հաշվարկները: changeCopy(PayrollRegister object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const PayrollRegister&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: hours={120,168,150,180}, p=1500: Ինդեքսները սկսվում են 0-ից:

№12 Առաքման պատվերներ

Տվյալներ: Ստեղծել DeliveryOrders դաս՝ պահելով պատվերների առաքման հեռավորությունների զանգվածը և p դրամ սկզբնական վճարը: Վճարն է p+100·distance դրամ:

Հաշվարկներ: Հաշվել առաքումների ընդհանուր վճարը, գտնել ամենաթանկ առաքման ինդեքսը և 10 կմ-ից հեռու պատվերների քանակը:

Թույլատրելի արժեքներ: հեռավորությունները ≥0, p≥0:

Ֆունկցիաներ: analyze(const DeliveryOrders&) արտածում է երեք հաշվարկները: changeCopy(DeliveryOrders object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const DeliveryOrders&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: distances={2,12,5,18}, p=500: Ինդեքսները սկսվում են 0-ից:

№13 Կինոթատրոնի ցուցադրություններ

Տվյալներ: Ստեղծել CinemaSchedule դաս՝ պահելով ցուցադրությունների վաճառված տոմսերի քանակների զանգվածը և դահլիճի ընդհանուր p նստատեղերի թիվը:

Հաշվարկներ: Հաշվել բոլոր ցուցադրությունների չվաճառված տեղերի ընդհանուր թիվը, գտնել ամենաշատ տոմս վաճառած ցուցադրության ինդեքսը և առնվազն 80 տոկոս լցված ցուցադրությունների քանակը:

Թույլատրելի արժեքներ: p-ն դրական ամբողջ թիվ է, վաճառված տոմսերի քանակները ամբողջ են և 0-ից p միջակայքում:

Ֆունկցիաներ: analyze(const CinemaSchedule&) արտածում է երեք հաշվարկները: changeCopy(CinemaSchedule object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const CinemaSchedule&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: tickets={40,80,100,60}, p=100: Ինդեքսները սկսվում են 0-ից:

№14 Սրճարանի հաշիվներ

Տվյալներ: Ստեղծել CafeBills դաս՝ պահելով սեղանների պատվերների գումարների զանգվածը և p սպասարկման տոկոսը: Վերջնական հաշիվն է amount·(1+p/100):

Հաշվարկներ: Հաշվել վերջնական հաշիվների գումարը, գտնել ամենափոքր վերջնական հաշվով սեղանի ինդեքսը և 20000 դրամից մեծ վերջնական հաշիվների քանակը:

Թույլատրելի արժեքներ: պատվերների գումարները >0, 0≤p≤100:

Ֆունկցիաներ: analyze(const CafeBills&) արտածում է երեք հաշվարկները: changeCopy(CafeBills object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const CafeBills&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: amounts={8000,25000,14000,30000}, p=10: Ինդեքսները սկսվում են 0-ից:

№15 Բջջային սակագնի օգտագործում

Տվյալներ: Ստեղծել MobileUsage դաս՝ պահելով բաժանորդների օգտագործած րոպեների զանգվածը և փաթեթում ներառված p րոպեները: Գերազանցող յուրաքանչյուր րոպեն արժե 20 դրամ:

Հաշվարկներ: Հաշվել լրացուցիչ վճարների գումարը՝ Σmax(0,minutes-p)·20, գտնել ամենամեծ լրացուցիչ վճարով բաժանորդի ինդեքսը և սահմանաչափը գերազանցած բաժանորդների քանակը:

Թույլատրելի արժեքներ: րոպեները և p-ն ոչ բացասական ամբողջ թվեր են:

Ֆունկցիաներ: analyze(const MobileUsage&) արտածում է երեք հաշվարկները: changeCopy(MobileUsage object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const MobileUsage&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: minutes={80,120,200,50}, p=100: Ինդեքսները սկսվում են 0-ից:

№16 Ավտոկայանատեղիի այցելություններ

Տվյալներ: Ստեղծել ParkingVisits դաս՝ պահելով այցելությունների ժամաքանակների զանգվածը և մեկ սկսված ժամի p դրամ վճարը: Գումարը ceil(hours)·p է:

Հաշվարկներ: Հաշվել բոլոր այցելությունների վճարների գումարը, գտնել ամենաբարձր վճարով այցելության ինդեքսը և 2 ժամից ավելի տևած այցելությունների քանակը:

Թույլատրելի արժեքներ: ժամերը >0, p>0: ceil-ը դեպի վեր կլորացումն է մինչև ամբողջ թիվ:

Ֆունկցիաներ: analyze(const ParkingVisits&) արտածում է երեք հաշվարկները: changeCopy(ParkingVisits object, int index, double value) փոփոխում և արտածում է

միայն տեղային պատճենը: reversed(const ParkingVisits&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: hours={0.5,2.2,1,4.1}, p=300: Ինդեքսները սկսվում են 0-ից:

№17 Դասընթացի արդյունքներ

Տվյալներ: Ստեղծել CourseResults դաս՝ պահելով ուսանողների ամփոփիչ գնահատականների զանգվածը և p անցողիկ շեմը:

Հաշվարկներ: Հաշվել խմբի միջին գնահատականը, գտնել ամենաբարձր գնահատականով ուսանողի ինդեքսը և p-ից ոչ փոքր գնահատական ստացած ուսանողների քանակը:

Թույլատրելի արժեքներ: գնահատականները և p-ն իրական թվեր են 0-ից 100 միջակայքում:

Ֆունկցիաներ: analyze(const CourseResults&) արտածում է երեք հաշվարկները: changeCopy(CourseResults object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const CourseResults&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: scores={55,80,45,90}, p=60: Ինդեքսները սկսվում են 0-ից:

№18 Ամսական բաժանորդագրության ծախսեր

Տվյալներ: Ստեղծել SubscriptionUsage դաս՝ պահելով հաճախորդների օգտագործած ծառայության միավորների զանգվածը և ամսական p անվճար սահմանաչափը: Յուրաքանչյուր ավել միավորն արժե 50 դրամ:

Հաշվարկներ: Հաշվել գերազանցման ընդհանուր վճարը՝ Σmax(0,usage-p)·50, գտնել ամենափոքր օգտագործմամբ հաճախորդի ինդեքսը և անվճար սահմանաչափը չգերազանցած հաճախորդների քանակը:

Թույլատրելի արժեքներ: օգտագործումները և p-ն ոչ բացասական ամբողջ թվեր են:

Ֆունկցիաներ: analyze(const SubscriptionUsage&) արտածում է երեք հաշվարկները: changeCopy(SubscriptionUsage object, int index, double value) փոփոխում և արտածում է միայն տեղային պատճենը: reversed(const SubscriptionUsage&) վերադարձնում է նոր օբյեկտ՝ շրջված զանգվածով և նույն p-ով: Արտածել բնօրինակը յուրաքանչյուր կանչից հետո:

Փորձնական տվյալներ: usage={20,55,40,80}, p=50: Ինդեքսները սկսվում են 0-ից:

Հանձնման պահանջներ

1. Ներկայացնել նշանակված տարբերակի ամբողջական լուծումը և C++ սկզբնաղբյուրի ֆայլը:
2. Կցել առնվազն երեք փորձարկման մուտքային տվյալներն ու արդյունքները:
3. Ներկայացնել կարճ եզրակացություն և պատրաստ լինել բացատրելու լուծումը:
4. Ֆայլային առաջադրանքի դեպքում կցել նաև անհրաժեշտ մուտքային և ելքային ֆայլերը, իսկ նախագծային առաջադրանքի դեպքում՝ նախագծման նյութերը: